

An End-to-End MLOps System for Vietnamese License Plate Recognition: From Synthetic Training to Monitored Production Serving

Nguyen Dinh Dan, Hoang Xuan Son, Nguyen Hoang Thai, Nguyen Van Vu
Group 4, DDM501 — AI in Production: From Models to Systems
FPT School of Business and Technology (FSB), Vietnam

Abstract—Automatic license plate recognition (ALPR) is a mature computer-vision task, yet deployed recognizers fail silently when lighting, weather or cameras change. This paper presents an end-to-end machine learning operations (MLOps) system for Vietnamese license plates that treats the model as one component of a monitored, reproducible lifecycle. A YOLOv8 detector and a HOG+SVM character classifier are served through a FastAPI service that always loads the model version tagged @production in an MLflow registry. Apache Airflow retrains the model weekly behind a data-quality gate and a champion/challenger promotion rule; Prometheus, Grafana and Alertmanager monitor system health and label-free input and prediction drift through twelve custom metrics and fifteen alert rules; a continuous-integration pipeline enforces 80% test coverage and an end-to-end quality gate. The deployed classifier reaches 92.93% character accuracy and 93.35% macro-F1 on a synthetic test set and reads 85.0% of synthetic plates exactly, but only 35.7% (15/42; 95% CI 23–51%) of manually labelled real images. The evaluation and monitoring stack localized this gap to motorbike and blue plates and to the image-processing path, rejected four weaker retrained models, and detected a real cold-start latency fault. We describe the design in enough detail to be reproduced, verify each lifecycle capability with a concrete scenario, assess the system against the ML Test Score, and report the engineering failures that the MLOps practices exposed and the limitations that remain.

Index Terms—MLOps, license plate recognition, model registry, drift monitoring, continuous integration, responsible AI

I. INTRODUCTION

Parking lots, toll stations, residential compounds and gated factories in Vietnam still record license plates largely by hand. A guard reads the plate, types it into a logbook or spreadsheet and compares it with the plate at exit. The process is slow at peak hours, error-prone at night and in rain, and requires staff around the clock. ALPR is a well-studied answer to this problem [1], [2], but a recognizer that scores well offline is not yet a dependable system. Input distributions drift with lighting, weather, dirt and camera replacement, and in production there are no labels to reveal the degradation. Sculley *et al.* [3] showed that the model is a small part of a real ML system and that most risk lies in data dependencies, configuration and monitoring; surveys of industrial deployments confirm that operating a model is harder than training it [4], [5].

This project therefore targets the *lifecycle* rather than the strongest possible model. We ask three research questions:

- **RQ1** (engineering): can a small team build a lifecycle in which data, training, registry, serving, monitoring and

retraining are connected by explicit, automated quality gates, using only open-source components on one machine?

- **RQ2** (evaluation): how large is the gap between the synthetic benchmark used for training and real Vietnamese plate images, and where in the pipeline do the errors originate?
- **RQ3** (operations): which failures do MLOps practices actually catch that ordinary testing would not?

The contributions are:

- a containerized system of twelve services covering data validation, training, experiment tracking, a model registry with alias-based promotion, serving, monitoring, alerting and scheduled or drift-triggered retraining;
- a shared champion/challenger rule that promotes a model only above a quality gate *and* when it is not worse than the serving model, with hot reload and one-step rollback;
- label-free drift signals (input brightness, input resolution and the rate of outputs that match plate grammar) wired into Prometheus alerts, a Grafana dashboard and an Airflow monitoring DAG, plus a simulation toolkit that reproduces seven traffic and drift scenarios;
- an evaluation on 42 manually labelled real images that quantifies the synthetic-to-real gap and attributes the errors to pipeline stages and plate groups;
- an account of six silent failures that the MLOps practices turned into visible, fixed defects, and a self-assessment against the ML Test Score [14].

Section II reviews related work and Section III defines the problem. Sections IV–VI describe the architecture, the ML pipeline and the MLOps lifecycle. Section VII evaluates model and system, Section VIII covers responsible AI, and Sections IX–X discuss lessons, limitations and future work. The appendix lists the commands needed to reproduce the results.

II. RELATED WORK

License plate recognition. Classical ALPR pipelines detect the plate, segment characters and classify them [1]. Histograms of oriented gradients (HOG) [6] with support vector machines (SVM) [7] remain a strong CPU-only baseline for isolated characters, while single-stage detectors such as YOLO [8],

[9] are the common choice for plate localization. Deep segmentation-free recognizers such as CRNN [10] and the unconstrained-scenario pipeline of Silva and Jung [2] achieve higher accuracy but need large labelled real datasets and, usually, a GPU. Contrast-limited adaptive histogram equalization (CLAHE) [11] is a standard pre-processing step under uneven lighting. Vietnamese plates add specific difficulties: two-line motorbike plates with a two-character series and a hyphen, several background colours (white, yellow, blue) and a restricted alphabet.

MLOps. Kreuzberger *et al.* [12] define MLOps as the combination of continuous integration, delivery and training with experiment tracking, a model registry and monitoring. Google’s maturity model [13] distinguishes manual deployment (level 0), automated training pipelines (level 1) and automated pipeline delivery (level 2). The ML Test Score [14] lists 28 data, model, infrastructure and monitoring tests needed for production readiness, and MLflow [15] is a widely used open-source tracking and registry tool. Amershi *et al.* [4] and Paleyes *et al.* [5] report that data management, deployment and monitoring dominate the effort in practice.

Drift and responsible AI. Concept and data drift are surveyed in [16], [17]; Rabanser *et al.* [18] argue for detecting shift from unlabelled inputs, which motivates our label-free signals. SHAP [19] and LIME [20] provide global and local explanations, and model cards [21] encourage reporting performance per subgroup.

III. PROBLEM DEFINITION AND REQUIREMENTS

A. Task and scope

The system receives a vehicle or plate image through a REST API and returns the plate string. A Vietnamese plate string s is valid if it consists of a two-digit province code, a series letter, an optional second series character and three to five digits, e.g., 51F12345 or 29AB1234. The scope is one- and two-line plates, one plate per still image. Real-time video tracking, multiple plates per image and foreign plates are out of scope.

B. Stakeholders and use cases

The primary users are parking and gate operators, who need a plate string per vehicle within a few seconds; the secondary users are the ML engineers and operators who maintain the service, who need to know when quality degrades and how to roll back. Three use cases drive the design: (U1) an entry camera posts a full scene; (U2) an existing detector posts a pre-cropped plate; (U3) an operator investigates an alert, inspects the serving model and, if needed, rolls back.

C. Requirements and success metrics

Table I lists the prioritized requirements with their implementation status, and Table II the success metrics on three levels: business, model and system. Targets were fixed before the real-image evaluation, and two of them are not met; we report them as such.

TABLE I
PRIORITIZED REQUIREMENTS AND STATUS

ID	Requirement	Status
FR1	Recognize the plate in an uploaded image via REST API	Done
FR5	Monitor system and model through dashboards	Done
NFR1	p95 latency below 3 s per image	Not met
FR2	Accept full scenes and pre-cropped plates	Done
FR3	Expose health and model information	Done
NFR3	Reproducible containerized deployment	Done
FR4	Track experiments and model versions	Done
NFR2	Alerting on failure and drift	Done
NFR4	CI/CD with quality gates	Done
NFR5	Explainability and fairness analysis	Done

TABLE II
SUCCESS METRICS: TARGETS AND MEASURED VALUES

Level	Metric	Target	Measured
Business	Exact plate, 42 real images	> 70%	35.7% (not met)
Business	Throughput, one client	> 10/min	≈64/min
Model	Character accuracy	> 90%	92.93%
Model	Character macro-F1	> 90%	93.35%
Model	Digit/letter accuracy gap	< 10%	1.0%
System	p95 latency, real images	< 3 s	≈4.1 s (not met)
System	Test coverage	> 80%	81.8%
System	Time to alert, API down	< 5 min	≈75 s

IV. SYSTEM ARCHITECTURE

A. Components

Fig. 1 shows the system and Table III its services. Twelve containers run on one Docker Compose network: ten long-running services, each with a health check, and two one-shot initializers (MinIO bucket creation and Airflow database migration). A thirteenth service, the `trainer`, is started on demand through a Compose profile. Each service has a single responsibility, so a failure is contained: if MLflow is unavailable the API keeps serving its last loaded model, and if the API is down Prometheus still records `up=0` and raises an alert.

B. Request flow

A prediction request passes through six steps: (1) a middleware applies a sliding-window rate limit per client IP and answers HTTP 429 with a `Retry-After` header when it is exceeded; (2) the upload size is checked against a 10 MB limit (HTTP 413); (3) the image is decoded in memory, and invalid images are rejected with HTTP 400; (4) the input brightness and width are recorded as drift metrics; (5) recognition runs in a thread pool so that `/health` and `/metrics` stay responsive during long inferences; and (6) prediction count, latency, character count, detection confidence and the outcome of the plate-grammar check are recorded before the JSON response is returned. The response carries the corrected plate text, the raw classifier output, the plate type, the number of characters, the detection confidence, a plate-format score, the latency and a timestamp.

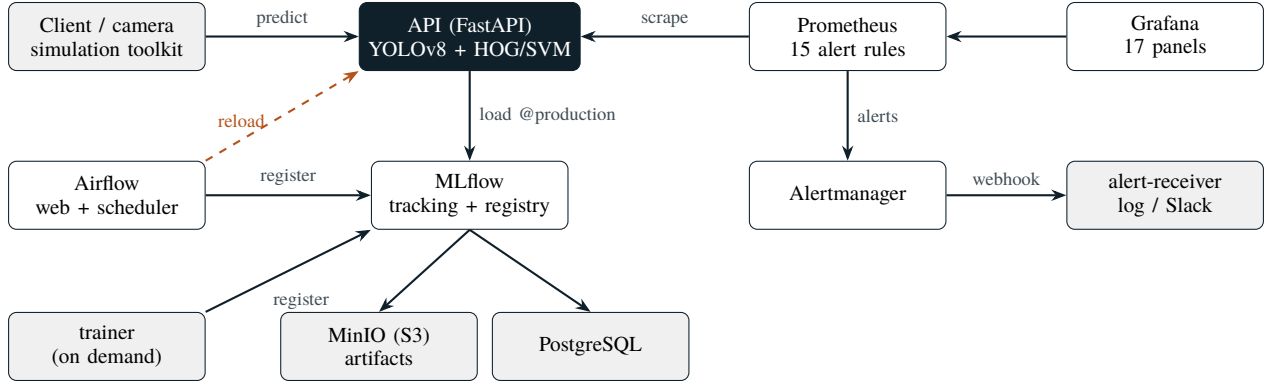


Fig. 1. System architecture. Solid arrows are data or control flow; the dashed arrow is the hot reload issued after a promotion. The monitoring DAG in Airflow also queries Prometheus (not drawn).

TABLE III
SERVICES OF THE DOCKER COMPOSE STACK

Service	Port	Role
postgres	5432	Metadata store for MLflow and Airflow
minio	9000/1	S3-compatible artifact store
minio-init	—	One-shot: creates the artifact bucket
mlflow	5000	Tracking server, registry, artifact proxy
api	8000	FastAPI inference service
trainer	—	On-demand training job (profile train)
prometheus	9090	Metrics scraping and alert rules
alertmanager	9093	Alert grouping, routing, inhibition
alert-receiver	9094	Webhook sink, log, optional Slack
grafana	3000	Provisioned dashboard
airflow-init	—	One-shot: database migration, admin user
airflow-web.	8080	Airflow UI and REST API
airflow-sched.	—	DAG scheduling (LocalExecutor)

C. Technology choices

Docker Compose was preferred to Kubernetes because the system runs on a single machine for this course; the API is stateless and loads its model from the registry, so replicas behind a load balancer need no code change. HOG+SVM was preferred to a deep OCR network because it runs on CPU, yields a small model, trains in seconds and can be explained with SHAP and LIME; the OCR stage is isolated behind one interface, so a stronger recognizer can replace it without touching the MLOps layer. MinIO provides S3-compatible artifact storage proxied by MLflow, so only the MLflow server holds storage credentials. Airflow was chosen over cron because it records every run, retries failed tasks and exposes a REST API that the monitoring DAG uses to trigger retraining.

V. ML PIPELINE

A. Inference stages

Each request runs seven stages.

(1) *Enhancement*. The image is converted to grayscale and enhanced with CLAHE [11] (clip limit 2.0, 8×8 tiles) followed

by a bilateral filter ($d = 9$, $\sigma = 75$) that removes noise while keeping character edges.

(2) *Detection*. A YOLOv8n model [9] fine-tuned on plates returns the plate box with a confidence threshold of 0.25. When the weights are absent or detect nothing, a fallback finds rectangular contours whose aspect ratio matches Vietnamese plates. Requests may set `assume_plate_crop` to skip detection for pre-cropped input (use case U2).

(3) *Rectification*. The crop is rectified by a perspective transform, deskewed by the dominant text angle, and both 0° and 180° orientations are evaluated.

(4) *Plate type*. The aspect ratio $\rho = w/h$ decides between a one-line ($\rho > 2.5$) and a two-line plate; the type controls how characters are split into rows.

(5) *Segmentation*. For several enhanced variants of the crop (CLAHE with clip 3.0 and 4×4 tiles, bilateral filter $d = 5$, $\sigma = 45$), Otsu binarization is applied in normal and inverted polarity. Connected components are filtered by area, height (10–82% of the plate height), width and aspect ratio (< 1.9); each candidate binarization is scored by the number of plausible components, their fill ratio and penalties for huge or border-touching blobs. Characters are sorted by row and column and normalized to 32×32 pixels.

(6) *Classification*. HOG features are computed with 4×4-pixel cells, 2×2-cell blocks and 9 orientation bins. A 32×32 character has 8×8 cells and 7×7 overlapping blocks, so the descriptor has

$$D = 7 \cdot 7 \cdot (2 \cdot 2 \cdot 9) = 1764 \quad (1)$$

dimensions (the alternative 8×8-pixel cells give 3·3·36 = 324). Features are standardized, $z = (x - \mu)/\sigma$, and classified by an SVM with the radial basis function kernel,

$$f(\mathbf{z}) = \sum_i \alpha_i y_i \exp(-\gamma \|\mathbf{z}_i - \mathbf{z}\|^2) + b, \quad (2)$$

combined one-versus-one over 31 classes: the ten digits and the 21 letters used on Vietnamese plates (I, J, O, Q and W are excluded).

(7) *Plate grammar*. Each position has a character type: positions 1–2 must be digits, position 3 a letter and the

TABLE IV
GRAMMAR-BASED CORRECTION MAPS

Expected	Correction
Digit	B→8, G→6, I/L/T→1, O→0, S→5, Z→2
Letter	0→G, 1→L, 4→A, 5→S, 6→G, 8→B

tail digits. The classifier’s scores are first constrained to the allowed classes per position; remaining confusions are corrected with fixed maps (Table IV). The final text is matched against the valid-plate pattern, and a plate-format score (0–20) summarizes how well the candidate fits. Among all crop variants, orientations and binarizations, the candidate with the highest score is returned; evaluation stops early when a valid plate scores at least 12, and a wider fallback search runs when the best score is below 10.

B. Training data

Real labelled character crops were not available, so the classifier is trained on synthetic characters. For each of the 31 classes the generator renders 250 clean glyphs and 400 “plate-style” glyphs from TrueType fonts. Plate-style glyphs are perturbed with Gaussian blur, erosion or dilation, additive noise and random thresholding (thresholds 35–95), which imitates low-resolution, over- or under-exposed plates. This gives $31 \times 650 = 20,150$ samples, split 80/20 with stratification (16,120 training and 4,030 test characters). Generation is deterministic: seeds are fixed and the same fonts are installed in every container image, so two runs on different machines produce identical data.

Before training, a data gate computes a validation report (sample count, class count, minimum and maximum samples per class, imbalance ratio, presence of NaN, pixel range) and stops the pipeline with a non-retryable failure on NaN values, pixels outside $[0, 255]$ or fewer than 31 classes.

C. Real-image evaluation set

To measure real performance, the team labelled the plate text of 50 sample images from a public plate-detection dataset. Eight images were unreadable even for a human (blur, occlusion, extreme angle) and were excluded and listed separately, leaving 42 labelled images. Each label records a confidence level, the vehicle type (car or motorbike), the line count and the plate colour, so the results can be broken down by group. Ground-truth plate boxes from the dataset were also cropped, which lets us run the OCR stage alone and separate detection errors from recognition errors.

D. Evaluation metrics

Character accuracy is the share of correctly classified test characters. Macro-F1 averages the per-class F1 score with equal weight per class,

$$F1_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{2P_k R_k}{P_k + R_k}, \quad (3)$$

TABLE V
FEATURE × CLASSIFIER COMPARISON (SYNTHETIC SPLIT)

Feature	Classifier	Accuracy	Macro-F1	Fit (s)
HOG-324	SVM	94.12	94.15	0.6
HOG-324	Logistic	93.43	93.44	82.3
HOG-1764	Random forest	92.74	92.79	1.9
HOG-324	Random forest	92.51	92.57	1.4
HOG-1764	Logistic	92.51	92.52	518.4
HOG-1764	SVM (untuned)	89.63	90.18	5.3
HOG-324	KNN	89.98	90.13	0.0
Wavelet	Random forest	89.06	89.11	1.0
Raw	Random forest	88.71	88.75	1.1
Raw	SVM	88.48	88.63	2.3
Raw	Logistic	88.02	88.05	363.6
HOG-1764	KNN	87.79	87.96	0.0
Wavelet	Logistic	86.06	86.14	75.8
Wavelet	SVM	85.83	85.91	0.7
Raw	KNN	83.76	83.97	0.0
Wavelet	KNN	79.49	79.78	0.0

where P_k and R_k are precision and recall of class k and $K = 31$; it penalizes weak rare classes that accuracy would hide. Exact-plate accuracy counts a plate as correct only if every character is correct, which is the business metric. For small samples we report the Wilson score interval [23] for a proportion $\hat{p}$ over n trials,

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad (4)$$

with $z = 1.96$ for 95% confidence.

E. Experiments and model selection

Table V compares sixteen feature/classifier combinations on a fixed synthetic split using scikit-learn [22]: four feature extractors (raw pixels, Haar wavelets, HOG-324 and HOG-1764) and four classifiers (SVM, logistic regression, random forest and k -nearest neighbours). HOG features dominate raw pixels and wavelets, and the SVM is the strongest classifier for the 324-dimensional HOG variant. Five-fold cross-validation of the top configuration gives a macro-F1 of 0.936 ± 0.057 . Logistic regression is competitive in accuracy but slower to fit by two to three orders of magnitude, which matters for weekly retraining.

The two finalists were retrained on the full data (Table VI). The 1764-dimensional model was tuned with three-fold grid search over C and γ (best $C = 5$, $\gamma = \text{auto} = 1/D$, CV macro-F1 0.921). Although HOG-324 is better per character, HOG-1764 reads more *whole plates* correctly on a 60-plate synthetic benchmark. The finer cells keep stroke details that distinguish confusable pairs within a plate, while HOG-324 wins on isolated glyphs. Because the business objective is the full plate, HOG-1764 was deployed. This decision illustrates why the model metric alone should not drive promotion.

VI. MLOPS LIFECYCLE

A. Experiment tracking

Every training run, manual or orchestrated, logs to MLflow: parameters (feature method, classifier, hyper-parameters, sam-

TABLE VI
FINALISTS TRAINED ON THE FULL SYNTHETIC SET

Metric	HOG-1764 (deployed)	HOG-324
Character accuracy (4,030 chars)	92.93%	96.90%
Character macro-F1	93.35%	—
Exact plate, 60 synthetic plates	85.0%	81.7%
End-to-end character accuracy	96.4%	93.6%
Exact plate with YOLO detection	83.3%	—

Algorithm 1 Champion/challenger registration and promotion

Require: trained pipeline m , test accuracy a , metadata μ

- 1: $v \leftarrow \text{REGISTER_VERSION}(m, \mu)$ ▷ always registered
- 2: $a^* \leftarrow \text{CHAMPION_ACCURACY}(@\text{production})$
- 3: **if** $a < a_{\min}$ **then**
- 4: tag v as rejected (gate); **return**
- 5: **else if** $a^* \neq \emptyset$ **and** $a < a^*$ **then**
- 6: tag v as rejected (champion); **return**
- 7: **end if**
- 8: $\text{SET_ALIAS}(\text{production}, v)$
- 9: $\text{POST}(/model/reload, \text{admin token})$ ▷ hot reload

ple counts, test size, feature dimension), metrics (accuracy, macro-F1, cross-validation mean and standard deviation), tags (trigger reason: scheduled, manual or drift) and artifacts (classification report, confusion matrix and a metadata file). Run metadata lives in PostgreSQL and artifacts in MinIO, so the history survives container restarts.

B. Model registry

Each model is registered as a single scikit-learn Pipeline(scaler $\rightarrow$ SVM) under the registered name `lpr-ocr-classifier`, together with metadata (feature method, feature dimension, class list, scikit-learn version). Packaging scaler and classifier together makes training-serving skew in feature scaling impossible, and the metadata lets the API fail fast if the model expects a feature dimension the code does not produce. Serving resolves the alias `@production`; MLflow stages are deprecated in favour of aliases. Because the MLflow 2.17 `get_model_info` call rejects alias URIs, the shared registry module first resolves the alias to a concrete version.

Promotion follows one shared rule, implemented once and used by both the Airflow DAG and manual training:

$$\text{promote} \iff a \geq a_{\min} \wedge (a^* = \emptyset \vee a \geq a^*), \quad (5)$$

where a is the challenger accuracy, $a_{\min} = 0.90$ and a^* is the accuracy logged by the run behind `@production`. Algorithm 1 shows the full procedure. Rollback is a single alias change followed by an authenticated `POST /model/reload`; no container restart is needed, and the previous version remains in the registry.

C. Orchestration

Two Airflow DAGs run on the LocalExecutor with PostgreSQL as metadata database. Table VII lists their tasks.

TABLE VII
AIRFLOW DAG TASKS

Task	Purpose
<i>lpr_training_pipeline</i> (@weekly)	
<code>generate_data</code>	Render 20,150 synthetic characters (fixed seed)
<code>validate_data</code>	Data gate: NaN, pixel range, 31 classes, imbalance
<code>extract_features</code>	HOG-1764 descriptors
<code>train</code>	Standardize, 3-fold CV (macro-F1), fit SVM, log to MLflow
<code>evaluate</code>	Accuracy, macro-F1, report, confusion matrix
<code>register_model</code>	Register version, apply rule (5)
<code>reload_api</code>	Hot reload, only after a promotion
<i>lpr_monitoring_pipeline</i> (every 15 min)	
<code>check_api_health</code>	/health and model loaded
<code>check_model_consistency</code>	Served version = @production
<code>canary_prediction</code>	Known image returns expected plate
<code>query_prometheus</code>	Success rate, p95, errors, drift ratios
<code>evaluate_health_and_drift</code>	Compare with thresholds
<code>write_monitoring_report</code>	JSON report per run
<code>branch_on_drift</code>	Trigger retraining or no action
<code>fail_if_unhealthy</code>	Make an unhealthy run visible as failed

TABLE VIII
MONITORING DAG THRESHOLDS (ENVIRONMENT-CONFIGURABLE)

Check	Threshold
Recognition success rate	≥ 0.5
p95 prediction latency	≤ 3.0 s
Error rate	$\leq 0.1/s$
Brightness drift ratio r_b	≤ 0.3
Width drift ratio r_w	≤ 0.5
Minimum predictions for statistical checks	20

`lpr_training_pipeline` runs weekly and is unpaused at creation, so scheduled retraining starts without manual action. Tasks pass small metadata dictionaries through XCom and large arrays through a per-run directory. Each task retries twice with a delay, but quality-gate violations raise a non-retryable failure so that bad data is not retried into a model. `lpr_monitoring_pipeline` runs every 15 minutes. It checks API health, verifies that the version served by the API equals the version behind `@production` (detecting a missed reload), sends a canary image with a known plate, evaluates PromQL thresholds (Table VIII) and writes a JSON report. When drift is detected and `AUTO_RETRAIN_ON_DRIFT` is enabled, a branch triggers the training DAG with the reason `drift`. Statistical checks are skipped when fewer than 20 predictions arrived in the window, because a p95 over three requests is noise.

D. Serving

Table IX lists the endpoints. The OpenAPI specification documents every request and response field with examples and every error code with a typed error body. Security and robustness measures are: uploads limited to 10MB; a limit of 60 requests per minute per client; a token, compared in constant time, for the reload endpoint; error responses that

TABLE IX
REST ENDPOINTS

Method	Path	Purpose
GET	/	Service information and links
GET	/health	Liveness and model readiness
GET	/model/info	Version, source, feature method, metrics
POST	/predict	Multipart image, optional assume_plate_crop
POST	/predict/base64	Base64-encoded JPEG or PNG
POST	/model/reload	Load current @production (X-Admin-Token)
GET	/metrics	Prometheus exposition format

never contain stack traces or internal paths; a non-root user in a multi-stage image; and a check that refuses to load a Git LFS pointer file in place of model weights. A warm-up inference at start-up and after every reload moves the one-off initialization of the detector and feature extractor off the request path. If MLflow cannot be reached, the API falls back to the local model files and reports `model_source: local` in `/model/info`, which makes the degraded mode visible.

E. Monitoring

The API exports twelve custom metrics (Table X), scraped by Prometheus every 10 s. Because production traffic is unlabelled, model quality is observed through proxies. *Data drift* is measured on the input brightness b and width w by comparing a recent window with a baseline,

$$r_b = \left| 1 - \frac{\bar{b}_{1h}}{\bar{b}_{7d}} \right|, \quad r_w = \left| 1 - \frac{\bar{w}_{1h}}{\bar{w}_{7d}} \right|, \quad (6)$$

where each mean is computed in PromQL as the ratio of the histogram's `_sum` and `_count` rates. A night-time camera lowers $\bar{b}$; a camera replacement changes $\bar{w}$. *Prediction drift* is measured by the share of outputs that match the plate grammar,

$$s = \frac{N_{\text{valid}}}{N_{\text{valid}} + N_{\text{invalid}}}, \quad (7)$$

which falls when inputs move away from the training distribution even though no label is available. The Grafana dashboard provisions 17 panels in three rows: service health (request rate, latency percentiles and heatmap, errors, status codes, uptime, model information), model behaviour (predictions by plate type, detection confidence, characters per plate) and drift (success rate, brightness and width against their baselines, in both production and fast windows).

F. Alerting

Table XI lists the fifteen alert rules in five groups. Each rule has a `for` duration so that a single slow request does not page anyone. The drift rules come in two variants: the production rules compare one hour with seven days, while the fast rules compare five minutes with one hour so that a live simulation can fire them during a demonstration. The `NoPredictions` rule sums over plate types, because a plate type that simply did not occur would otherwise fire the alert while traffic flows.

TABLE X
CUSTOM PROMETHEUS METRICS EXPORTED BY THE API

Metric (type)	Meaning
<code>api_requests_total</code> (C)	Requests by method, endpoint, status
<code>api_request_latency_seconds</code> (H)	End-to-end request latency
<code>model_predictions_total</code> (C)	Predictions by plate type
<code>model_prediction_latency_seconds</code> (H)	Inference latency
<code>model_prediction_errors_total</code> (C)	Errors by type (incl. rate limit)
<code>model_info</code> (G)	Loaded version and source
<code>plate_char_count</code> (H)	Characters per plate
<code>detection_confidence</code> (H)	YOLO confidence
<code>plate_recognition_result_total</code> (C)	Valid vs. invalid plate format
<code>plate_format_score</code> (H)	Grammar fit score, 0–20
<code>input_image_brightness</code> (H)	Mean grayscale brightness
<code>input_image_width_pixels</code> (H)	Input width

C: counter, H: histogram, G: gauge.

Alertmanager groups notifications by alert name and severity. Critical alerts are sent after 10 s and repeated hourly; informational alerts are batched. An inhibition rule suppresses warning and informational alerts while `APIDown` fires, because every API metric is meaningless then. All notifications go to a webhook receiver that logs them, keeps the last 50 in a queryable endpoint, appends them to a JSON-lines file and optionally forwards them to Slack.

G. Simulation toolkit

To exercise monitoring without a real camera, a simulation toolkit replays images against the API under seven scenarios: normal traffic, nightfall (gradual darkening), camera swap (resolution change), mixed weather (motion blur, noise, JPEG compression, occlusion), prediction drift (non-plate images), a rate-limit spike and malformed inputs. Drift is produced by deterministic image transforms, so each scenario has a known expected alert. The toolkit has its own unit tests.

H. Testing and continuous integration

The suite contains 149 tests in five categories (Table XII) and covers 81.8% of `src/` and `api/`. Tests use small fixtures and mocks for MLflow and Prometheus, so they run without the Docker stack.

GitHub Actions runs three dependent jobs on every push and pull request (Table XIII). The end-to-end benchmark is a *model* gate inside CI: a change to pre-processing or segmentation that lowers plate accuracy on the synthetic benchmark fails the build even if every unit test passes. A separate manual workflow builds and pushes images to the GitHub Container Registry. Model binaries are stored with Git LFS and cached in CI to save bandwidth.

I. Reproducibility

Reproducibility is enforced at four levels. *Code*: dependencies are pinned to exact versions (e.g., scikit-learn 1.5.2, scikit-image 0.24.0, NumPy 1.26.4), the same versions are used in the trainer and the API, and the version is stored in the model

TABLE XI
PROMETHEUS ALERT RULES

Group	Alert	Condition	Severity	For
api	APIHighErrorRate	prediction errors > 0.1/s over 5 min	warning	2 m
api	APIHighLatency	p95 request latency > 5 s over 5 min	warning	3 m
api	APIDown	scrape target lpr-api down	critical	1 m
model	HighPredictionLatency	p95 inference latency > 3 s	warning	5 m
model	LowCharacterDetection	median characters per plate < 5	warning	10 m
model	LowDetectionConfidence	median YOLO confidence < 0.5	warning	10 m
model	NoPredictions	total prediction rate over 15 min = 0	info	30 m
drift	LowRecognitionSuccessRate	valid-format share $s < 0.5$ over 30 min	warning	15 m
drift	InputBrightnessDrift	$r_b > 0.3$ (1 h vs. 7 d)	warning	30 m
drift	InputResolutionDrift	$r_w > 0.5$ (1 h vs. 7 d)	warning	30 m
drift (fast)	InputBrightnessDriftFast	$r_b > 0.3$ (5 min vs. 1 h)	warning	2 m
drift (fast)	InputResolutionDriftFast	$r_w > 0.5$ (5 min vs. 1 h)	warning	2 m
drift (fast)	LowRecognitionSuccessRateFast	$s < 0.5$ over 5 min with traffic	warning	2 m
drift (fast)	RateLimitBurst	more than 5 HTTP 429 responses in 2 min	warning	—
system	PrometheusTargetDown	any scrape target down	critical	2 m

TABLE XII
TEST SUITE BY CATEGORY

Category	What is tested
Unit	Pre-processing, features, classifier, detection, pipeline, grammar, registry module, crop script
Integration	All endpoints, auth, size and rate limits, metrics, MLflow fallback, error bodies
Data quality	Synthetic data schema, value ranges, class coverage and balance
Model validation	Accuracy and macro-F1 thresholds, robustness to noisy inputs
Operations	Monitoring DAG logic, alert receiver, simulation transforms and simulator
End-to-end	Full pipeline on labelled synthetic plates

TABLE XIII
CONTINUOUS INTEGRATION JOBS

Job	Steps and gates
lint	ruff and black on all Python packages, DAGs and tests
test	pytest with coverage gate 80%; E2E benchmark gate: character accuracy $\geq 85\%$, exact plate $\geq 50\%$; reports uploaded as artifacts
docker-build	Build API, MLflow, Airflow and trainer images

metadata. *Data*: synthetic data is generated from fixed seeds and fonts, and the real-image labels are versioned as CSV files with an explicit list of excluded images. *Models*: every model is an immutable registry version with its run, parameters and artifacts. *Environment*: the whole stack is described by one Compose file and a `.env.example`; secrets are never committed.

VII. EVALUATION

A. Recognition accuracy (RQ2)

Fig. 2 contrasts the synthetic benchmark with the real images. On the 42 real images the full pipeline reads 15 plates

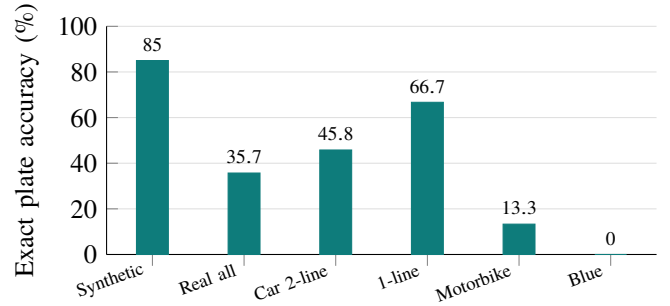


Fig. 2. Synthetic-to-real gap and per-group exact plate accuracy on real images (1-line and blue groups contain only 3 and 2 images).

exactly (35.7%; 95% Wilson interval (4) 23–51%) with 63.2% character accuracy. Table XIV breaks the result down. Two-line car plates fare best; motorbike plates (two-character series and a hyphen on the top line) and blue plates with white text fail most. The blue-plate failure is a polarity problem: the segmentation is tuned for dark text on a light background.

Feeding ground-truth plate crops raises character accuracy from 63.2% to 81.0% (exact plate 42.9%). Roughly half of the character errors therefore originate in detection crops, rectification and deskew rather than in the classifier, which tells us where to invest next. Images from the detector’s training and validation splits score similarly (37.0% vs. 33.3%), suggesting no strong detector overfitting on this sample.

B. Error analysis

Inspecting the failures gives four recurring causes. (i) *Merged or broken characters*: on low-resolution or dirty plates two characters merge into one component or a character splits in two, shifting all later positions. (ii) *Confusable glyphs*: 8/B, 5/S, 2/Z and 0/D, consistent with the explanation analysis in Section VIII; grammar correction fixes these only where the position type is known. (iii) *Row assignment on motorbike plates*: the hyphen and the two-character series move components between rows. (iv) *Inverted polarity* on blue plates.

TABLE XIV
REAL-IMAGE RESULTS BY GROUP (FULL PIPELINE)

Group	Exact plate	Share
All images	15/42	35.7%
Two-line car plates	11/24	45.8%
Motorbike plates	2/15	13.3%
One-line plates	2/3	—
Yellow plates	4/8	50.0%
White plates	11/32	34.4%
Blue plates	0/2	—
Detector train / val split	10/27 / 5/15	37.0 / 33.3%
Ground-truth crops (OCR only)	18/42	42.9%

TABLE XV
MODEL REGISTRY OUTCOMES

Ver.	Source	Outcome	Accuracy
v1	HOG-324, manual	not promoted	96.90% [†]
v2	HOG-1764 tuned, manual	@production	92.93%
v3	Airflow DAG	rejected (champion)	90.19%
v4	Airflow, scheduled	rejected (champion)	90.06%
v5	Trainer smoke test	rejected (gate)	87.63%
v6	Airflow, new stack	rejected (champion)	90.32%

[†]Measured on that run's own test split; v1 was registered before the deployment decision in Section V.

Causes (i), (iii) and (iv) lie before the classifier, which agrees with the ground-truth-crop experiment.

C. Serving performance

Sequentially replaying the real images (84 requests, one client, rate limit lifted) gives a median latency of 0.50 s, a p95 of 4.07 s and about 64 images per minute. The distribution is long-tailed: easy plates return after the first candidate, while hard images exhaust the early-exit condition and evaluate many crop variants and binarizations. The p95 therefore misses the 3 s requirement. Before the warm-up fix, the first request after a restart took 2.6–4.9 s and the monitoring DAG once observed a p95 in the >10 s bucket; after the fix the first request took 1.1 s.

D. Lifecycle behaviour (RQ1)

Table XV lists the registry. Three scheduled or manual DAG runs produced challengers at about 90% accuracy; each was registered but rejected by (5) because the champion scores 92.93%. The DAG trains with untuned hyper-parameters ($C = 10$, $\gamma = \text{scale}$), which explains why its challengers are consistently weaker than the tuned champion, and the rule prevented each of them from silently replacing it. A trainer smoke test with a reduced data set (87.63%) was stopped by the quality gate. Table XVI summarizes the scenarios used to verify each lifecycle capability on the running stack.

E. ML Test Score self-assessment

Table XVII maps the system to the four sections of the ML Test Score [14]. The assessment is our own and is meant to show coverage and gaps, not to produce a certified score.

TABLE XVI
VERIFICATION OF LIFECYCLE CAPABILITIES ON THE RUNNING STACK

Capability	Scenario	Observed
Registry-driven serving	Start API with MLflow up	/model/info: v2, source MLflow
Degraded mode	MLflow unreachable	Local fallback, model_source: local
Rollback	Alias v2→v1→v2 + reload	Effective immediately, no restart
Scheduled retraining	Weekly DAG run	v4, v6 registered, rejected
Quality gate	Trainer smoke run	v5 rejected below 0.90
Consistency check	Monitoring DAG	Served version = @production
Failure alert	Stop API container	APIDown after ≈75 s, resolved on recovery
Latency fault	Restart, first request	Detected by DAG; fixed by warm-up

TABLE XVII
SELF-ASSESSMENT AGAINST THE ML TEST SCORE

Section	Evidence in this system	Level
Data	Data gate, data-quality tests, deterministic generation, no personal data stored	Yes
Model	CV, threshold tests, champion baseline, per-group real-image slices (offline)	Partial
Infrastructure	Pinned versions, E2E gate in CI, registry rollback, canary in monitoring DAG	Yes
Monitoring	Version consistency, input and prediction drift, latency, errors, alerts	Yes

VIII. RESPONSIBLE AI

Fairness. The task has no human sensitive attribute, so we measure whether accuracy is even across input groups that correlate with users. On synthetic data the digit/letter gap is 1.0% (91.6% vs. 92.6%) and the weakest class is G. Real images reveal a vehicle-type disparity: motorbike plates are read correctly 13% of the time versus 46% for cars, so motorbike owners, who are the majority of road users in Vietnam, would bear a higher error risk. Mitigations are additional motorbike and coloured-plate data and per-group monitoring, in the spirit of model cards [21].

Explainability. SHAP [19] ranks HOG cells by their global influence per class, and LIME [20] explains individual predictions by perturbing image regions. Together with the confusion matrix they identify the confusable pairs 8/B, 5/S, 2/Z and 0/D, which is also where grammar correction is applied.

Privacy. A plate can be linked to its owner and is treated as personal data under Vietnam's Decree 13/2023/ND-CP [24]. Images are decoded in memory and neither images nor plate strings are stored; metrics and logs contain no plate text, and administrative endpoints require a token. A deployment that must retain records would need a stated purpose, a retention period and access control.

Ethics and human oversight. ALPR can enable surveillance

TABLE XVIII
SILENT FAILURES AND THE PRACTICE THAT EXPOSED THEM

Failure	Exposed by	Fix
Model pickled with scikit-learn 1.9, served with 1.5	Tests differing across environments	Pinned versions, trainer image
Metadata claimed 1764-d features; model used 324-d	/model/info audit	Fail fast on mismatch
Weekly retraining never ran (DAG paused by default)	Airflow audit	Unpaused at creation
Manual training could replace a better model	Code review	Shared rule (5)
Old models unreadable after moving artifacts to MinIO	model_source: local	Artifact migration
First request after restart took up to >10s	Monitoring DAG	Warm-up inference

of individuals. At the measured real-image accuracy the system must only assist; a human must confirm any enforcement decision, and the plate-format score can be used to route low-confidence results to manual review.

IX. DISCUSSION

A. Lessons learned (RQ3)

Table XVIII lists failures that left the system running but wrong. None of them crashed the service, and none was caught by a unit test alone. Each was surfaced by an MLOps practice: environment parity, a model information endpoint, a DAG audit, code review of the promotion path, or the monitoring DAG. This supports Sculley *et al.*'s observation that ML systems fail silently [3], and it answers RQ3: the practices that matter most are those that compare what *is* running with what *should* be running.

B. Maturity

In Google's terms [13] the system reaches level 1: training is automated and triggered by schedule or drift, models are validated and registered automatically, and serving is monitored. It does not reach level 2, because new pipeline code is built and tested by CI but not deployed to a target environment automatically.

C. Threats to validity

Internal validity: champion and challenger are compared on their own test splits rather than on a fixed benchmark, so a lucky split could favour a challenger. *External validity*: 42 real images from one public dataset do not represent all cameras, regions and weather; the Wilson interval is wide. *Construct validity*: the label-free success rate measures grammatical validity, not correctness, so a model that outputs valid but wrong plates would pass it. *Labelling*: one annotator labelled the real images; ambiguous images were excluded, which may bias accuracy upwards.

D. Limitations

The classifier is trained only on synthetic glyphs; the p95 latency target is not met; rate limiting is held in memory per API instance; and the stack runs on one host with single-instance Prometheus and no high availability.

X. CONCLUSION AND FUTURE WORK

We built and operated an end-to-end MLOps system for Vietnamese license plate recognition in which data, training, registry, serving, monitoring and retraining form one closed loop with explicit quality gates (RQ1). The system is reproducible and measurable, and its evaluation showed that synthetic accuracy (85.0% exact plates) is an upper bound for real images (35.7%), with about half of the character errors originating before the classifier (RQ2). The practices that caught the most defects were those comparing running state with intended state (RQ3).

Future work, in priority order:

- 1) motorbike-specific segmentation and inverted-polarity handling for blue plates;
- 2) training on real character crops derived from the labelled images, and a larger, multi-annotator real test set;
- 3) promotion decided on a fixed real-image benchmark instead of per-run splits, and aligning the DAG hyperparameters with the tuned configuration;
- 4) a latency budget for the candidate search to meet the p95 target;
- 5) Kubernetes deployment with shared rate limiting, redundant monitoring and automated delivery to reach maturity level 2.

TEAM CONTRIBUTIONS

N. D. Dan: real-image labelling and evaluation, synthetic benchmark and end-to-end tests, registry-to-serving loop, reproducibility, API hardening, drift alerts, Alertmanager. *H. X. Son*: inference pipeline (plate grammar for two-letter series, 1/7 disambiguation, latency reduction, thread safety) and API rate limiting. *N. H. Thai*: MinIO and multi-container Airflow, monitoring DAG, traffic and drift simulation toolkit, demo guide. *N. V. Vu*: initial system, CI/CD, Airflow training DAG, YOLOv8 integration, real sample dataset.

APPENDIX

REPRODUCING THE RESULTS

The commands below, run from the repository root, start the stack, train a model on demand and reproduce the evaluation. Credentials are read from `.env`, created from `.env.example`.

```
git lfs pull
cp .env.example .env
docker compose up -d --build
docker compose --profile train run --rm trainer
pytest --cov=src --cov=api
python scripts/evaluate_lpr_end_to_end.py
python simulations/run_simulation.py
```

The services are then available at `localhost:8000/docs` (API), `:5000` (MLflow), `:8080` (Airflow), `:9090` (Prometheus), `:9093` (Alertmanager) and `:3000` (Grafana).

REFERENCES

- [1] S. Du, M. Ibrahim, M. Shehata, and W. Badawy, “Automatic license plate recognition (ALPR): A state-of-the-art review,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 23, no. 2, pp. 311–325, 2013.
- [2] S. M. Silva and C. R. Jung, “License plate detection and recognition in unconstrained scenarios,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 580–596.
- [3] D. Sculley *et al.*, “Hidden technical debt in machine learning systems,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 2503–2511.
- [4] S. Amershi *et al.*, “Software engineering for machine learning: A case study,” in *Proc. IEEE/ACM Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2019, pp. 291–300.
- [5] A. Paleyes, R.-G. Urma, and N. D. Lawrence, “Challenges in deploying machine learning: A survey of case studies,” *ACM Comput. Surv.*, vol. 55, no. 6, pp. 114:1–114:29, 2022.
- [6] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2005, pp. 886–893.
- [7] C. Cortes and V. Vapnik, “Support-vector networks,” *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.
- [8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.
- [9] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [10] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, 2017.
- [11] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in *Graphics Gems IV*. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.
- [12] D. Kreuzberger, N. Kühl, and S. Hirschl, “Machine learning operations (MLOps): Overview, definition, and architecture,” *IEEE Access*, vol. 11, pp. 31866–31879, 2023.
- [13] Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning,” 2020. [Online]. Available: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- [14] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, “The ML test score: A rubric for ML production readiness and technical debt reduction,” in *Proc. IEEE Int. Conf. Big Data*, 2017, pp. 1123–1132.
- [15] M. Zaharia *et al.*, “Accelerating the machine learning lifecycle with MLflow,” *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.
- [16] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Comput. Surv.*, vol. 46, no. 4, pp. 44:1–44:37, 2014.
- [17] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, 2019.
- [18] S. Rabanser, S. Günnemann, and Z. C. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019.
- [19] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.
- [20] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 1135–1144.
- [21] M. Mitchell *et al.*, “Model cards for model reporting,” in *Proc. Conf. Fairness, Accountability, Transparency (FAT*)*, 2019, pp. 220–229.
- [22] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [23] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.
- [24] Government of Vietnam, “Decree No. 13/2023/ND-CP on personal data protection,” 2023.